Sobrecarga sintáctica de Java (clases, main, tipos) en estudiantes de primer ciclo para presentar artículos

Brigith de los Angeles Toledo Jumbo, https://orcid.org/0009-0005-7918-2293
Universidad Nacional de Loja
Loja, Ecuador
brigith.toledo@unl.edu.ec

Resumen— El presente ensayo analiza artículos para comprender la dificultad que tienen los estudiantes de primer ciclo en conocer la sintaxis inicial de Java y la resolución de problemas. Ya que elementos como clases, main y tipos podrían añadir esfuerzo mental durante las primeras actividades. Para ello, se realizó una revisión de cinco publicaciones, donde se comparó sus métodos, resultados y aportes. Los trabajos destacan la utilidad de los ejemplos resueltos, la orientación paso a paso y la combinación de distintas medidas para evaluar el esfuerzo mental. Además, se concluye que es necesario estudiar directamente como influye la sintaxis inicial de Java en estos estudiantes y evaluar el uso de plantillas que luego se retiraran conforme el avance del aprendizaje.

Palabras clave— Carga cognitiva, Java, programación introductoria, sintaxis inicial, estudiantes de primer ciclo, complejidad del código, ejemplos resueltos, plantillas de programación.

I. INTRODUCCIÓN

Aprender a programar implica relacionar las instrucciones escritas con los cambios que se producen durante la ejecución. En estudiantes de primer ciclo, generar una salida correcta no garantiza tener un modelo mental sólido sobre variables, memoria y flujo de control. Por ello, la enseñanza introductoria debe atender tanto a la escritura del código como a la comprensión de su funcionamiento.

En Java un simple mensaje como “Hola Mundo”, puede provocar que un estudiante se encuentra con una clase, un método, modificadores, un tipo de entorno y un parámetro. Aquí es donde surge el problema de determinar si escribir o interpretar simultáneamente estos elementos añade un esfuerzo que dificulte la concentración en el objetivo lógico, además de establecer qué elementos convienen explicar antes del inicio, cuáles pueden proporcionarse mediante una plantilla y como introducir después su significado.

La importancia de este estudio radica en la relación con la capacidad limitada de la memoria de trabajo y la interacción entre los elementos que el estudiante debe procesar, ya que la dificultad depende de sus conocimientos previos: lo que para un programador experimentado es una estructura familiar, para un estudiante son varios componentes complejos e independientes.

Por lo tanto, se tiene como objetivo analizar como los artículos investigados contribuyen a sustentar el estudio de la carga cognitiva asociada a la sintaxis inicial de Java. Para lo cual se busca:

- Identificar aportes sobre la relación entre la complejidad del código y el esfuerzo mental.
- Examinar enfoques de medición útil en la programación introductoria.
- Comparar ayudas didácticas y sus límites
- Delimitar qué aspectos requieren evidencia específica en estudiantes de primer ciclo.

Las contribuciones esperadas de este ensayo es una síntesis de los artículos con sus resultados comprobados para aclarar mejor el tema propuesto.

II. METODOLOGÍA

Se investigo 5 artículos relacionados con el tema, con la ayuda de las herramientas ACM DL e IEEE Xplore. Para la búsqueda se utilizó la cadena: (Java) AND ("novice programmers") AND ("cognitive load" OR "syntax difficulties") AND (CS1). Los datos utilizados de cada artículo fueron sus objetivos, características metodológicas, hallazgos reportados y limitaciones. De este contenido se extrajo aportes de cada autor para realizar el análisis respectivo, lo que dicen los autores se presenta en los resultados. Posteriormente se da una discusión según la investigación dada, para luego escribir una conclusión y colocar las referencias.

III. RESULTADOS

A. *Sintaxis y resolución de problemas*

Según Sands **[1]** los principiantes deben procesar reglas sintácticas y aplicar conceptos a problemas nuevos, y la combinación de ambas exigencias aumenta el riesgo de verse agobiados. En sus palabras: “Por separado, la sintaxis y el uso contribuyen a la carga cognitiva del alumno. Si se combinan, aumenta el riesgo de que el alumno se sienta abrumado.”, afirma el problema general.

Sands dice que los ejemplos resueltos deben organizarse mediante modelado, acompañamiento y retirada gradual del apoyo. El estudiante observa una solución, reflexiona sobre sus decisiones y completa partes cada vez mayores. El autor advierte que ejemplos complejos pueden dividir la atención y que un apoyo útil para novatos puede resultar innecesario cuando el estudiante ya posee los conocimientos previos.

Según el mismo autor, la programación en parejas distribuye actividades entre conductor y navegante, mientras el modelado en vivo hace visible el diseño, la implementación, las pruebas y la depuración. Estos beneficios dependen de preparar la colaboración y adaptar la tarea.

B. *Métricas y esfuerzo*

Según Thorgeirsson y Vahrenhold **[2]** las líneas de código predijeron la carga cognitiva mejor que las otras métricas examinadas. No confirmaron que DepDegree, una medida de dependencia de datos, fuera el predictor más fuerte. Las valoraciones comparativas de expertos mostraron una mayor habilidad para prever lo que iba a pasar durante los análisis exploratorios.

Según estos autores, la evidencia viene de un estudio con 551 estudiantes, donde cada participante resolvió seis tareas aleatorias de 24 fragmentos disponibles en su lenguaje elegido. La carga se evaluó con la escala de esfuerzo mental de Pass y subescalas del NASA-TLX. La comparación no demuestra que eliminar líneas provoque menor carga (se refiere a predicción en tareas de comprensión bajo condiciones controladas).

C. *Medición complementaria*

Según Suryani et al. **[3]** la carga cognitiva es un estado interno que se estima mediante indicadores indirectos, todos con ventajas y limitaciones. Su revisión recupero inicialmente 424 publicaciones, donde 85 pasaron a evaluación de calidad y 71 quedaron incluidas (35 de congreso y 36 de revista).

De las 71 publicaciones, 28 combinaron diferentes formas de medir

la carga cognitiva. En 10 de esas 28 se utilizaron tanto cuestionarios sobre el esfuerzo percibido como datos del desempeño, por ejemplo, errores o tiempo de resolución. Esta fue la combinación más frecuente. La revisión identifica instrumentos como NASA-TLX, Paas y Klepsch.

Los autores señalan que el tiempo y los errores pueden verse afectados por factores distintos de la carga cognitiva. Una interfaz dividida en pasos puede reducir la exigencia mental y aumentar el tiempo de navegación. Por eso, un resultado lento o incorrecto no basta para diagnosticar la sobrecarga. También advierten dificultades para aislar tipos de carga, muestras pequeñas y problemas de generalización.

D. Efectos selectivos

Según Noviello et al. **[4]** las animadas generadas por IA pueden producir beneficios selectivos en el aprendizaje inmediato. En su estudio de Java, con 151 estudiantes, la diferencia estimada de desempeño fue de 5,420 a favor de la animación, con p ajustado = 0,017. En ciclos "while", con 138 estudiantes, fue de 8,227, con p ajustado = 0,015. Son diferencias en la puntuación del estudio, no porcentajes relativos de mejora.

De acuerdo con los autores, no hubo una diferencia fiable en carga cognitiva agrupada en Java (diferencia estimada = 0,004, p ajustado = 0,974). Tampoco se identificaron diferencias significativas en los resultados posteriores de examen analizados en ambas instituciones. En Python, el análisis inmediato agrupado con 961 estudiantes, no mostró una mejora fiable. Los tamaños de muestra variaron según la actividad y el resultado.

Los autores analizaron las instituciones por separado porque diferían en lenguaje, cobertura temática y organización. Por eso sus resultados no permiten atribuir las diferencias entre instituciones exclusivamente a Java o Python, y los análisis por perfiles de participación fueron exploratorios.

E. Tutoría contextual

Según Su et al. **[5]**, un tutor inteligente centrado en el aprendizaje debe orientar mediante pistas y preguntas, en lugar de entregar soluciones. El sistema clasifica errores de compilación, recupera recursos específicos del curso mediante RAG y utiliza explicaciones verificadas por docentes para contextualizar la respuesta.

Conforme a los autores, el IDE mínimo y las respuestas breves se diseñaron con la intención de reducir demandas cognitivas innecesarias. Los autores reportan pruebas de intentos de manipular las instrucciones del tutor y describen los rechazos de soluciones en esas pruebas. No proporcionan una evaluación con estudiantes que calcule la reducción de carga, frustración o mejora del aprendizaje.

TABLA I
APORTES DE LOS ARTICULOS AL ESTUDIO DE LA SINTAXIS INICIAL DE JAVA

Autor	**Aporte reportado**	**Relación con clases, main y tipos**
Sands (2019)	Según Sands, aprender sintaxis y aplicar constructos a problemas nuevos simultáneamente puede abrumar al principiante. Propone ejemplos resueltos, colaboración y modelado en vivo.	Fundamenta medir los componentes iniciales y retirar el apoyo gradualmente.
Thorgeirsson y Vahrenhold (2026)	Según los autores, las líneas de código predijeron la carga cognitiva mejor que otras métricas examinadas; las valoraciones de expertos mostraron una capacidad predictiva superior.	Justifica contrastar las características del código con medidas del esfuerzo del estudiante.
Suryani et al. (2024)	Según los autores, combinar métodos ayuda a compensar las limitaciones de cada indicador; 28 de 71 publicaciones utilizaron enfoques multimodales.	Fundamenta combinar esfuerzo percibido, desempeño y registros del proceso.
Noviello et al. (2026)	Según los autores, las animaciones generaron beneficios inmediatos selectivos, sin una reducción global robusta de carga cognitiva ni mejoras en los exámenes posteriores examinados.	Permite distinguir facilidad inmediata, esfuerzo y aprendizaje duradero al evaluar una ayuda.
Su et al. (2026)	Según los autores, el tutor integra clasificación de errores, recursos recuperados y preguntas orientadoras; su interfaz mínima y respuestas breves buscan evitar demandas innecesarias.	Orienta el diseño de ayuda contextual para errores iniciales de Java.

Esta tabla muestra que los artículos investigados aportan fundamentos importantes que permiten justificar una evaluación en estudiantes de primer ciclo.

IV. DISCUSIÓN

Sands aporta el argumento más cercano a la justificación del tema donde los principiantes enfrentan al mismo tiempo la sintaxis y resolución de problemas. Thorgeirsson y Vahrenhold añaden que las características del código tienen relación con el esfuerzo, pero sus resultados predictivos no justifican una regla universal de "menos líneas, mejor aprendizaje". Suryani et al. explican por qué el esfuerzo no puede inferirse exclusivamente del rendimiento.

No toda exigencia sintáctica es necesariamente carga ajena al aprendizaje, ya que su función depende del objetivo de la tarea, los conocimientos previos y la representación. Si el objetivo es aprender tipos, las declaraciones son contenido central, pero si se quiere explorar una secuencia de instrucciones, su explicación extensa puede desplazar la atención.

Se propone comenzar con una estructura proporcionada y acompañada de preguntas breves, seguida de actividades que requieran reconstruir partes y explicar su función. Una plantilla vacía no equivale por sí sola a un ejemplo resuelto, ya que necesita una tarea y orientación que hagan comprensible la solución. La propuesta adapta el modelado, acompañamiento y retirada gradual de Sands, no reproduce una intervención sobre Java evaluada por él.

Noviello et al. Muestran que mejorar un resultado inmediato no implica reducir esfuerzo ni mejorar evaluaciones posteriores. Su et al. proponen evitar la entrega directa de soluciones, pero no demuestran todavía sus efectos con estudiantes. Por ello, el apoyo debe evaluarse por su contribución a la autonomía y a la comprensión, no solo por la cantidad de programas que permite terminar.

Además, se propone comparar escritura de la estructura tradicional con uso de una plantilla equivalente, manteniendo constantes las tareas, instrucciones, versión de Java, IDE y tiempo disponible. Se registrarían conocimientos previos, esfuerzo percibido, errores clasificados, tiempo y comprensión posterior sin apoyo. Los errores se separarían en estructura inicial, tipos, lógica y entorno.

Para distinguir escritura y comprensión, ambas condiciones deberían recibir la misma explicación conceptual. Si una condición

también recibe más orientación, se estarían comparando intervenciones compuestas. Una prueba inicial de experiencia y tareas posteriores comparables ayudarían a interpretar los resultados. La elección de una escala no convierte todos los indicadores en equivalentes ni permite identificar automáticamente la carga.

Existieron limitaciones en los cinco artículos debido a que se utilizó una transcripción de Sands y de Noviello donde no se verificaron todos los suplementos, también la revisión de Suryani et al. contiene discrepancias de las cuales se tomaron los principales.

V. CONCLUSIONES

La investigación de los artículos permitió identificar los fundamentos para organizar sintaxis y resolución de problemas, criterios para medir carga cognitiva y evidencia sobre los límites de las ayudas educativas. Sands aporta estrategias de apoyo temporal, en cambio, Thorgeirsson y Vahrenhold una evaluación de predictores del esfuerzo, y Suryani et al. una base para combinar medidas sin confundirlas con rendimiento.

Noviello et al. muestran que una mejora inmediata puede coexistir con ausencia de diferencias fiables en esfuerzo y exámenes posteriores. Su et al. aportan una arquitectura de apoyo contextual y preguntas guiadas, pero no una evaluación de efectos cognitivos con estudiantes.

En consecuencia, los trabajos justifican investigar la carga asociada a la estructura inicial de Java, pero no permiten afirmar que clases, main y tipos causen una sobrecarga en los estudiantes de primer ciclo.

REFERENCIAS

[1] P. Sands, "Addressing cognitive load in the computer science classroom," *ACM Inroads*, vol. 10, no. 1, pp. 44–51, Feb. 2019, doi: 10.1145/3210577.

[2] S. Thorgeirsson and J. Vahrenhold, "How (and How Not) Do Code Complexity Measures Predict Cognitive Load?," in *Proceedings of the 2026 ACM Conference on International Computing Education Research*, vol. 1, Aug. 2026, pp.15, doi: 10.1145/3765964.3811665.

[3] M. Suryani *et al.*, "Role, Methodology, and Measurement of Cognitive Load in Computer Science and Information Systems Research," *IEEE Access*, vol. 12, pp. 190007–190024, 2024, doi: 10.1109/ACCESS.2024.3514355.

[4] Y. Noviello *et al.*, "AI-Generated Traces for Novice Programmers: Learning Effects and Learner Differences in a Multi-Institutional Study," in *Proceedings of the 31st ACM Conference on Innovation and Technology in Computer Science Education* V. 1 (ITiCSE 2026), Jul. 2026, pp. 135-141, doi: 10.1145/3803400.3809346.

[5] L.-H. Su, Y. Gao, J. Morris, and S. Z. Atiq, "Learning-Centered Intelligent Tutor for Novice Java Programmers," in *Proceedings of the 57th ACM Technical Symposium on Computer Science Education* V. 2 (SIGCSE TS 2026), Feb. 2026, pp. 1541–1542, doi: 10.1145/3770761.3777266.